ARQUITECTURA DE SOFTWARE

1 POR CAPAS

Forma estructurada de organizar un programa colocandolas en niveles jerárquicos superpuestos, donde cada capa cumple una tarea específica y se comunica de manera unidireccional

CARACTERISTICAS CLAVE

1. Independencia jerárquica y modularidad estricta.
2. Abstracción de los detalles de bajo nivel respecto a la lógica de negocio superior

VENTAJAS

Mantenimiento simplificado, excelente para trabajo en equipo modular, alta reutilización de código y escalabilidad independiente de las capas

DESVENTAJAS

Posible sobrecarga de rendimiento por el paso de llamadas a través de múltiples capas y exceso de complejidad para aplicaciones demasiado pequeñas

PATRONES DE DISEÑO

SINGLETON
Garantiza que una clase tenga una única instancia en todo el ciclo de vida de la aplicación y proporciona un punto de acceso global a ella; ideal para compartir recursos pesados en proyectos grandes

FACTORY METHOD
Define una interfaz para crear un objeto, pero deja que las subclases decidan qué clase concreta instanciar; útil cuando un programa necesita gestionar múltiples creaciones de objetos dinámicamente

2 MONOLITICA

Modelo de diseño donde toda la aplicacion se construye como una unidad de codigo unico

ORGANIZACION

1. Usuario
2. Presentacion
3. Logica de negocio
4. Acceso a datos
5. Base de datos

VENTAJAS

Simplicidad inicial, despliegue unificado y pruebas directas l tener todo en un solo bloque

DESVENTAJAS

Escalabilidad limitada, alto riego ante fallos y dificultad de mantenimineto al crecer

PATRONES DE DISEÑO

ABSTRACT FACTORY
Crea familias de objetos relacionados o dependientes sin especificar sus clases concretas; garantiza compatibilidad en proyectos grandes

BUILDER
Permite la creación de objetos complejos paso a paso, separando el algoritmo de construcción de su representación final

PROTOTYPE
Crea nuevos objetos clonando instancias existentes (prototipos) para evitar costos altos de inicialización directa

3 CLIENTE SERVIDOR

En este modelo los roles están claramente separados: el Cliente solicita recursos, servicios o procesamiento, y el Servidor procesa la petición, interactúa con la base de datos y devuelve una respuesta estructurada

ORGANIZACION

- 2 Capas: Interfaz de usuario conectada directamente a la base de datos en la nube.
- 3 Capas: Separación física/lógica de Cliente, Servidor de Aplicaciones y Base de Datos.
- N Capas: Servicios independientes distribuidos para cada nivel de lógica o negocio específica.

VENTAJAS

Centralización de datos y seguridad, administración simplificada en el servidor, fácil mantenimiento y soporte de múltiples plataformas

DESVENJATAS

Alta dependencia del servidor (un fallo de infraestructura afecta a toda la red) y costos elevados de configuración técnica

PATRONES DE DISEÑO

ADAPTER
Funciona como un traductor o puente en el backend para permitir que clases o sistemas con interfaces incompatibles trabajen juntos sin alterar su código fuente.

FACADE
Proporciona una interfaz unificada y simplificada para un conjunto de subsistemas complejos, reduciendo el acoplamiento con el cliente

4 ARQUITECTURA MVC

Patrón arquitectónico clásico que desacopla la interfaz de usuario de la lógica interna en tres componentes independientes con responsabilidades únicas

ORGANIZACION

- Modelo: Gestiona los datos, el estado de la aplicación y la lógica de negocio esencial.
- Vista: Representa la interfaz gráfica y los elementos visuales con los que interactúa el usuario (Frontend).
- Controlador: Actúa como intermediario gestionando las entradas del usuario, actualizando el modelo y respondiendo con la vista adecuada

VENTAJAS

Excelente organización del código, permite que desarrolladores trabajen en paralelo (modelo, vista o controlador) sin interferir y garantiza una alta escalabilidad en aplicaciones medianas y grandes

DESVENTAJAS

Incremento considerable en la cantidad de archivos y necesidad rigurosa de estructurar las conexiones para evitar acoplamientos indeseados

PATRONES DE DISEÑO

DECORATOR
Permite añadir nuevas funcionalidades a un objeto de forma dinámica colocándolo dentro de objetos contenedores especiales, sin alterar el código original de la clase

PROXY
Proporciona un objeto sustituto o intermediario que controla el acceso al objeto real (por ejemplo, verificando permisos de seguridad del usuario antes de permitir una acción).

BRIDGE
Desacopla una abstracción de su implementación para que ambas puedan variar y evolucionar de forma totalmente independiente

ARQUITECTURA DE SOFTWARE

1 MICROSERVICIOS

Modelo arquitectónico que estructura una aplicación como un conjunto de servicios pequeños, independientes y autónomos que se comunican mediante APIs ligeras

ORGANIZACION

1. Cada microservicio posee su propia base de datos o esquema (desacoplamiento de datos).
2. Se despliegan, escalan y actualizan de forma completamente independiente.
3. Organizados en equipos pequeños y multifuncionales orientados a dominios de negocio (Domain-Driven Design).

VENTAJAS

- Alta escalabilidad modular (escalar solo el servicio que lo necesita).
- Flexibilidad tecnológica y despliegue continuo sin afectar a todo el sistema.
- Resiliencia: si un servicio falla, el resto de la aplicación puede seguir funcionando.

DESVENTAJAS

- Alta complejidad operativa y de red (latencia por llamadas distribuidas).
- Dificultad en la gestión de transacciones distribuidas y consistencia de datos.

PATRONES DE DISEÑO

COMPOSITE
Útil para componer estructuras de datos o componentes anidados en árbol dentro de un microservicio de forma uniforme

FLYWEIGHT
Permite optimizar el consumo de memoria compartiendo estados comunes y repetitivos dentro de microservicios con alta concurrencia o recursos limitados.

2 ORIENTADA A SERVICIOS

Estilo arquitectónico enfocado en la integración y reutilización de servicios empresariales a nivel corporativo utilizando protocolos de red compartidos

ORGANIZACION

1. Utiliza un Bus de Servicio Empresarial (ESB) centralizado; servicios de grano grueso; modelos de datos compartidos.

VENTAJAS

Excelente integración con sistemas heredados (legacy), reutilización masiva y gobernanza centralizada.

DESVENTAJAS

El ESB puede convertirse en un cuello de botella o punto único de fallo; acoplamiento más pesado.

PATRONES DE DISEÑO

MEDIATOR
Refleja el funcionamiento del Bus de Servicio Empresarial (ESB), centralizando y reduciendo el acoplamiento caótico en la comunicación entre múltiples servicios.

OBSERVER
Permite notificar de manera reactiva y desacoplada a múltiples servicios suscritos cuando ocurre un cambio o evento en el sistema.

3 HEXAGONAL

Patrón arquitectónico que aísla el núcleo de la aplicación (lógica de negocio o dominio) de los elementos tecnológicos externos.

ORGANIZACION

Núcleo central puro, Puertos (interfaces de entrada/salida) y Adaptadores (implementaciones tecnológicas concretas).

VENTAJAS

Alta testabilidad sin requerir bases de datos reales e independencia tecnológica total del núcleo.

DESVENJATAS

Incremento en la cantidad de clases, interfaces y capas de abstracción (mayor código inicial).

PATRONES DE DISEÑO

STRATEGY
Permite intercambiar dinámicamente diferentes algoritmos de negocio o reglas dentro del núcleo de la aplicación según el contexto o el adaptador que lo invoque.

STATE
Permite que un objeto del dominio altere su comportamiento interno de manera limpia cuando cambia su estado, facilitando la gestión de flujos complejos dentro de la lógica central.

ARQUITECTURA DE SOFTWARE

1 DIRIGIDA POR EVENTOS

Estilo arquitectónico centrado en la producción, detección, consumo y reacción ante eventos de negocio en tiempo real.

VENTAJAS

- Alto desacoplamiento entre componentes y procesamiento en tiempo real (asíncrono).
- Excelente escalabilidad horizontal para cargas de trabajo variables.

DESVENTAJAS

- Alta complejidad en el seguimiento, depuración y trazabilidad de los flujos de datos.
- Dificultad para garantizar la consistencia eventual entre microservicios/módulos.

ORGANIZACION

1. Componentes emisores (Producers), canal/broker de mensajes (Event Bus/Broker como Kafka o RabbitMQ) y componentes consumidores (Consumers) desacoplados.

PATRONES DE DISEÑO

COMMAND
Encapsula una petición como un objeto, lo que permite parametrizar clientes con diferentes peticiones, encolar o registrar solicitudes y soportar operaciones deshechables.

CHAIN OF RESPONSIBILITY
Permite pasar peticiones a lo largo de una cadena de manejadores; al recibir una petición, cada manejador decide procesarla o pasarla al siguiente en la cadena.

2 ARQUITECTURA SERVERLESS

Modelo de ejecución en la nube donde el proveedor gestiona dinámicamente la infraestructura y la asignación de recursos, cobrando únicamente por el tiempo de ejecución real

VENTAJAS

- Cero administración de servidores y escalado automático instantáneo desde cero.
- Reducción de costos operativos

DESVENTAJAS

- Latencia inicial por "arranque en frío".
- Dependencia del proveedor de nube (Vendor Lock-in) y limitaciones en tiempos máximos de ejecución.

ORGANIZACION

1. Funciones independientes e idempotentes (por ejemplo, AWS Lambda, Azure Functions) que se activan por disparadores (triggers) como peticiones HTTP, cambios en bases de datos o eventos.

PATRONES DE DISEÑO

ITERATOR
Proporciona una forma de acceder secuencialmente a los elementos de un objeto agregado sin exponer su representación subyacente.

MEMENTO
Permite capturar y externalizar el estado interno de un objeto para que pueda restaurarse a dicho estado posteriormente sin violar la encapsulación.

INTERPRETER
Define una representación gramatical para un lenguaje junto con un intérprete que utiliza la representación para interpretar sentencias en el lenguaje.

3 CLEAN ARCHITECTURE

Arquitectura en capas concéntricas centrada en la separación de responsabilidades y la regla de dependencia: el código interno nunca debe conocer nada del código externo

VENTAJAS

- Independencia total de frameworks, bases de datos, interfaces de usuario y agentes externos.
- Facilidad extrema para realizar pruebas unitarias (testability) de la lógica central del negocio.

DESVENJATAS

- Alta cantidad de código base inicial (boilerplate) y múltiples transformaciones de modelos entre capas.
- Curva de aprendizaje elevada para mantener estrictamente los límites arquitectónicos.

ORGANIZACION

Círculos concéntricos divididos en: Entidades, Casos de Uso, Adaptadores de Interfaz y Frameworks/Drivers (UI, Base de Datos, Web).

PATRONES DE DISEÑO

TEMPLATE METHOD
Define el esqueleto de un algoritmo en una operación, diferiendo algunos pasos a las subclases sin cambiar la estructura del algoritmo.

VISITOR
Permite definir una nueva operación en una jerarquía de clases sin cambiar las clases de los elementos sobre los que opera